DS IA-2 group 19

(Dynamic F1 Leaderboard System)

```
#include <iostream>
#include <string>
using namespace std;

// ======================= DRIVER NODE =======================
// Linked list node representing each driver
struct DriverNode {
    string name;
    string team;
    double lapTime;
    double totalTime;
    bool wrecked;
    DriverNode* next;

    DriverNode(string n="", string t="") {
        name = n;
        team = t;
        lapTime = 0;
        totalTime = 0;
        wrecked = false;
        next = nullptr;
    }
};

// ======================= INSERT DRIVER =======================
void insertDriver(DriverNode*& head, const string &name, const string &team) {
    DriverNode* node = new DriverNode(name, team);
```

```
    if (!head) {
        head = node;
        return;
    }
    DriverNode* cur = head;
    while (cur->next) cur = cur->next;
    cur->next = node;
}

// ======================== DISPLAY LEADERBOARD ========================
void displayLeaderboard(DriverNode* head, int lap, int totalLaps) {
    cout << "\n🏁 Leaderboard after Lap " << lap << "/" << totalLaps << " 🏁\n";
    cout << "Pos\tDriver\t\tTeam\t\tLapTime\tTotalTime\tStatus\n";
    cout << "-------------------------------------------------------\n";

    int pos = 1;
    DriverNode* cur = head;
    while (cur) {
        cout << "P" << pos++ << "\t" << cur->name << "\t\t"
            << cur->team << "\t\t"
            << cur->lapTime << "\t"
            << cur->totalTime << "\t\t"
            << (cur->wrecked ? " 💥 OUT" : "🏎 Racing") << '\n';
        cur = cur->next;
    }
    cout << "-------------------------------------------------------\n";
}

// ======================== REORDER DRIVERS ========================
// This function reorders drivers based on their times and wrecks
void adjustPositions(DriverNode*& head, bool showOvertakes = true) {
```

```
if (!head) return;

bool swapped;
do {
    swapped = false;
    DriverNode* prev = nullptr;
    DriverNode* curr = head;

    while (curr && curr->next) {
        DriverNode* nxt = curr->next;
        bool needSwap = false;

        // If current driver is wrecked, always push behind next
        if (curr->wrecked && !nxt->wrecked) needSwap = true;
        // Otherwise, compare times for racing drivers
        else if (!curr->wrecked && !nxt->wrecked && curr->totalTime > nxt->totalTime)
            needSwap = true;

        if (needSwap) {
            // Print overtakes only for on-track moves
            if (showOvertakes && !curr->wrecked && !nxt->wrecked) {
                cout << nxt->name << " is just behind " << curr->name
                    << "... he makes his move and slingshot pass "
                    << curr->name << " 📈 ! "
                    << nxt->name << " moves ahead!\n";
            }

            // Perform swap using pointers
            curr->next = nxt->next;
            nxt->next = curr;
            if (prev) prev->next = nxt;
```

```
            else head = nxt;


            prev = nxt;
            swapped = true;
        }
        else {
            prev = curr;
            curr = curr->next;
        }
    }
  } while (swapped);
}

// ======================= LAP HANDLER =======================
void nextLap(DriverNode*& head, bool safetyCar = false) {
  DriverNode* cur = head;
  while (cur) {
    if (!cur->wrecked) {
        cout << "Enter lap time for " << cur->name << ": ";
        cin >> cur->lapTime;
        if (safetyCar) cur->lapTime += 5.0;  // extra delay if safety car
        cur->totalTime += cur->lapTime;
    }
    cur = cur->next;
  }
  adjustPositions(head, true);  // overtakes on track
}

// ======================= PIT STOP =======================
void pitStop(DriverNode*& head, const string &driverName, double duration) {
  DriverNode* cur = head;
```

```
    while (cur) {
        if (cur->name == driverName && !cur->wrecked) {
            cout << cur->name << " enters pit stop for " << duration << "s 🛠";

            cur->totalTime += duration;

            // Collect who overtakes during this pit stop
            DriverNode* overt = cur->next;
            bool any = false;
            while (overt) {
                if (!overt->wrecked) {
                    if (!any) cout << " and meanwhile ";
                    cout << overt->name;
                    any = true;
                    if (overt->next) cout << ", ";
                }
                overt = overt->next;
            }
            if (any) cout << " overtake and gain track position!\n";
            else cout << " but holds position.\n";
            break;
        }
        cur = cur->next;
    }
    adjustPositions(head, false);  // reorder silently
}

// ======================= WRECK HANDLER =======================
void wreckCar(DriverNode*& head, const string &driverName) {
    DriverNode* cur = head;
    while (cur) {
```

```
        if (cur->name == driverName && !cur->wrecked) {
            cur->wrecked = true;
            cur->totalTime += 1000.0;  // push far back
            cout << " 💥 " << cur->name << " has crashed out!\n";
            cout << " 🚨 Safety Car deployed! Drivers behind slowed by +2s.\n";

            DriverNode* behind = cur->next;
            while (behind) {
                if (!behind->wrecked) behind->totalTime += 2.0;
                behind = behind->next;
            }
            break;
        }
        cur = cur->next;
    }
    adjustPositions(head, false);  // reorder silently
}

// ======================= MAIN =======================
int main() {
    DriverNode* head = nullptr;
    int n, totalLaps, currentLap = 0;

    cout << "Enter number of drivers: ";
    cin >> n;
    for (int i = 0; i < n; ++i) {
        string name, team;
        cout << "Enter driver " << i+1 << " name: ";
        cin >> name;
        cout << "Enter team: ";
        cin >> team;
```

```
    insertDriver(head, name, team);
}

cout << "Enter total laps: ";
cin >> totalLaps;

// Opening commentary
cout << "\n🎙 David Croft: Hello and welcome everyone to today's F1 Grand Prix!\n";
cout << "🎙 David Croft: I'm joined by the legend himself, Murray Walker.\n";
cout << "🎙 Murray Walker: Thank you Crofty! We're here on this beautiful circuit, ready for "
    << totalLaps << " laps of pure adrenaline.\n";
cout << "🎙 David Croft: The grid is set with these gladiator drivers -\n";
DriverNode* tmp = head;
while (tmp) {
    cout << " - " << tmp->name << " driving for " << tmp->team << "\n";
    tmp = tmp->next;
}
cout << "🎙 Murray Walker: The engines roars...\n";
cout << "🔥 It's Lights Out and Away We Go! 🔥 \n";

displayLeaderboard(head, currentLap, totalLaps);

while (true) {
    cout << "\n1. Next Lap\n2. Pit Stop\n3. Wrecked Car\n4. Safety Car Lap\n5. Exit\nChoice: ";
    int choice; cin >> choice;

    switch (choice) {
        case 1:
            if (currentLap < totalLaps) {
                ++currentLap;
                nextLap(head, false);
```

```
        displayLeaderboard(head, currentLap, totalLaps);
        if (currentLap == totalLaps) {
            cout << "\n🏁 We've got a winner! We've got a winner!! 🏁\n"
                << head->name << " has done it for " << head->team
                << " 🏆\n They are new F1 champions courtesy to "
                << head->name << "'s miraculous racing on track ✨\n"
                << "The crowd is on their feet, the team is overjoyed 🎉 🥳, "
                << "and the garage erupts in celebration 🙌 👏.\n"
                << "These are unforgettable moments in Formula 1 history! 🏎️🔥\n";
            return 0;
        }
    }
    break;

case 2: {
    string d; double dur;
    cout << "Enter driver for pit stop: "; cin >> d;
    cout << "Enter duration (s): "; cin >> dur;
    pitStop(head, d, dur);
    displayLeaderboard(head, currentLap, totalLaps);
    break;
}

case 3: {
    string d;
    cout << "Enter wrecked driver: "; cin >> d;
    wreckCar(head, d);
    displayLeaderboard(head, currentLap, totalLaps);
    break;
}
```

```
        case 4:
            cout << " 🚨 Safety Car Lap...\n";
            if (currentLap < totalLaps) {
                ++currentLap;
                nextLap(head, true);
                displayLeaderboard(head, currentLap, totalLaps);
                if (currentLap == totalLaps) {
                    cout << "\nWe've got a winner! We've got a winner!! "
                        << head->name << " has done it for " << head->team
                        << ". They are new F1 champions courtesy to "
                        << head->name << "'s miraculous racing on track. "
                        << "Wonderful moments on the track as we see the whole team celebrating the
much deserved victory 🎉 🏆 \n";
                    return 0;
                }
            }
            break;

        case 5:
            return 0;
    }
  }
}
```